Step 1 — The Match Report and the Matcher — Solution

> Back to the step · Hint 1 · Hint 2

Story: the typed shapes and the deterministic `match_skills` — the decision engine — built and proven with a fake retriever, no model or embeddings. **If not:** if the match logic or the score is wrong, a perfect semantic search still gives a wrong answer, and you'd be debugging RAG and arithmetic at once.

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `langchain-text-splitters`, `chromadb`, `openai`, `pydantic`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    match_cutoff: float
```

```
def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        match_cutoff=float(os.getenv("MATCH_CUTOFF", "0.8")),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes

```
# src/schemas.py
# RequiredSkills is what the job-description extractor returns; MatchReport is
# the final result the matcher produces.
from typing import List

from pydantic import BaseModel

class RequiredSkills(BaseModel):
    skills: List[str] = []

class MatchReport(BaseModel):
    score: int
    matched: List[str] = []
    missing: List[str] = []
```

The deterministic matcher (the decision engine)

```
# src/matcher.py
# For each required skill, ask a distance() function how far the closest resume
# chunk is; a distance at or below the cutoff counts as matched. distance() is
# passed in, so this whole decision is testable with no model or embeddings.
from src.schemas import MatchReport

def match_skills(required, distance, cutoff=0.8):
    matched = []
    missing = []
    for skill in required:
        if distance(skill) <= cutoff:
            matched.append(skill)
        else:
            missing.append(skill)
    total = len(required)
    if total == 0:
        score = 0
    else:
        score = round(100 * len(matched) / total)
    return MatchReport(score=score, matched=matched, missing=missing)
```

The tests (no model, no embeddings)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
```

```
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "JOBMATCH_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_matcher.py
# Run from the repo root: python -m tests.unit.test_matcher
from src.matcher import match_skills

# a fake distance(): small for skills the "resume" covers, big otherwise
COVERED = {"python": 0.1, "docker": 0.3, "aws": 0.4, "leadership": 0.5}

def fake_distance(skill):
    return COVERED.get(skill.lower(), 1.5)

def test_matched_and_missing():
    required = ["Python", "Docker", "AWS", "Kubernetes", "Leadership"]
    report = match_skills(required, fake_distance, cutoff=0.8)
    result = "OK"
    if report.missing != ["Kubernetes"]:
        result = "WRONG"
    if len(report.matched) != 4:
        result = "WRONG"
```

```
    print(f"case 1 (4 matched, 1 missing) -> {result}")

def test_score_is_a_percentage():
    required = ["Python", "Docker", "AWS", "Kubernetes", "Leadership"]
    report = match_skills(required, fake_distance, cutoff=0.8)
    result = "OK"
    if report.score != 80:
        result = "WRONG"
    print(f"case 2 (score = 80)           -> {result}")

def test_no_required_skills_is_zero():
    report = match_skills([], fake_distance)
    result = "OK"
    if report.score != 0:
        result = "WRONG"
    print(f"case 3 (empty -> score 0)     -> {result}")

if __name__ == "__main__":
    test_matched_and_missing()
    test_score_is_a_percentage()
    test_no_required_skills_is_zero()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (4 matched, 1 missing) -> OK
case 2 (score = 80)           -> OK
case 3 (empty -> score 0)     -> OK
```

The whole scoring decision is proven with made-up distances — so Step 3 only has to supply real ones from the resume search, not re-test the arithmetic.